

Cours 1 Programmation d'Applications Mobiles

Utilisation Android Studio (inscription effectuée, en attente de réponse)
Apprentissage de **Kotlin**

Mon 1^{er} programme en Kotlin

<https://play.kotlinlang.org/>

➔ Kotlin Playground

```
fun main() {  
    println("Hello, world!!!")  
}
```

« Hello, world » version Kotlin

main() -> Méthode principale

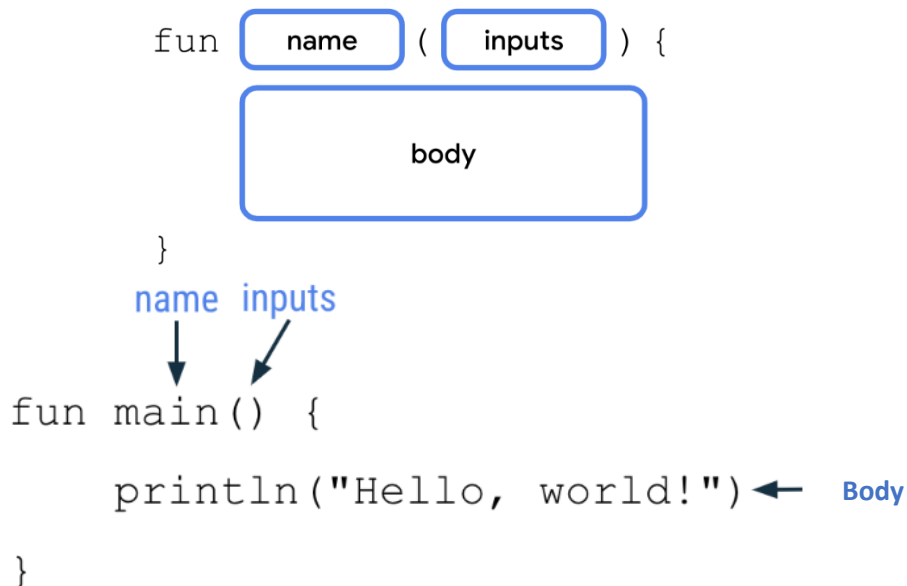
fun -> Fonction

println -> print + retour à la ligne

*/** */* ou *//* -> Commentaires

```
fun foo() {  
    println("Bonjour, monde !!!")  
}  
  
fun main(){  
    foo() /** Appel fonction foo() */  
}
```

Définir une fonction -> Lui donner un nom, l'ensemble des paramètres, l'ensemble des comportements qu'il peut avoir, et l'appel de la fonction



Inputs : On attend des données que l'on ne connaît pas (des variables typées) et qui peuvent être changées et qui vont être utilisées dans la fonction.

Faire en sorte que ça print 2 fois « Hello, Android » :

```
fun main() {  
    println("Hello, Android !!!") ;println("Hello, Android !!!")  
}
```

On va rarement utiliser les ';' en Kotlin.

```
fun main() {  
    println("Hello, Android !!!")  
    println("Hello, Android !!!")  
}
```

Boucle for (à voir pour la suite)

Normes à respecter :

space
↓
fun main() {
 println("Hello, world!")
}

indent by 4 spaces → fun main() {
 println("Hello, world!")
}

aligned vertically | fun main() {
 println("Hello, world!")
}

Réparer les erreurs de code :

```
fun main() {  
    println("Today is sunny!")  
}
```

```
! Syntax error: Expecting '"'.  
! Syntax error: Expecting ')'.  
}
```

Il faut traiter les erreurs du haut vers le bas (les erreurs du haut sont les plus importantes qui peuvent engendrer d'autres erreurs derrière)

```
fun main() {  
    println("Today is sunny!")  
}
```

En rajoutant « " » ça a retiré les 2 erreurs

Créer et utiliser des variables en Kotlin

Variable : Espace mémoire capable de stocker des valeurs à l'intérieur

Types de données :

- *String* -> Chaînes de caractères
- *Int* -> Entier
- *Double* -> Nombre décimal
- *Float* -> Flottant (nombre décimal plus précis)
- *Boolean* -> TRUE FALSE

```
fun main() {  
    val count: Int = 2 //Déclare une variable int nommée count et qui vaut 2 qui peut être utilisée plus tard  
    println(count)  
}
```

Schéma clé pour initialiser une valeur

val name : data type = initial value

val count : Int = 2

Utiliser une variable int dans un String

```
fun main() {  
    ! val count: Int = 2  
    println("You have count unread messages.")  
}
```

La variable count n'est pas utilisée, le string n'a pas fait appel à count

Pour faire appel à une variable on va utiliser le symbole « \$ »

```
fun main() {  
    val count: Int = 2  
    println("You have $count unread messages.")  
}
```

Type inference

Lorsque l'on déclare une variable, on peut ne pas toujours déclarer le type, qui peut être détecté automatiquement. Pas forcément une bonne idée, car cela peut créer des ambiguïtés.

val name = initial value

`val count: Int = 2`

Opérateurs Mathématiques basiques

Additions :

```
fun main() {  
    val unreadCount = 5  
    val readCount = 100  
    println("You have ${unreadCount + readCount} total messages in your inbox.")  
}
```

`${}` qui permet de sélectionner plusieurs variables pour effectuer nos calculs

```
fun main() {  
    val unreadCount = 5  
    val readCount = 100  
    println("You have $unreadCount + readCount total messages in your inbox.")  
}
```

Ne marche pas retourne « 5 + readcount »

```
fun main() {  
    val unreadCount = 5  
    val readCount = 100  
    println("You have $unreadCount + readCount total messages in your inbox.")  
}
```

Ne marche pas, retourne « 5 + 100 »

Soustractions :

```
fun main() {  
    val numberOfPhotos = 100  
    val photosDeleted = 10  
    println("$numberOfPhotos photos")  
    println("$photosDeleted photos deleted")  
    println("${numberOfPhotos - photosDeleted} photos left")  
}
```

```
100 photos  
10 photos deleted  
90 photos left
```

NB : Dans le nom de nos variables, on ne peut avoir de caractères de calcul comme +, -, *, /

Mettre à jour des variables

```
fun main() {  
    val cartTotal = 0  
    cartTotal = 20  
    println("Total: $cartTotal")  
}
```

Retourne une erreur en disant que la valeur ne peut pas être réassignée

Pour **val**, c'est une valeur qui n'aura qu'une seule affectation. Une fois initialisée, elle ne pourra plus bouger.
Pour avoir une variable qui va changer, on va utiliser **var**.

```

fun main() {
    !    var cartTotal = 0
        cartTotal = 20
        println("Total: $cartTotal")
}

```

Incrémenter et décrémenter des opérateurs

```

fun main() {
    var count = 10
    println("You have $count unread messages.")
    count = count + 1
    println("You have $count unread messages.")
}

```

Utilisation des autres types : Double

```

fun main() {
    !    val trip1: Double = 3.20
    !    val trip2: Double = 4.10
    !    val trip3: Double = 1.72
    val totalTripLength: Double = 0.0
    println("$totalTripLength miles left to destination")
}

```

Pour un double, mettre forcément une décimale avec « . »

```

fun main() {
    val trip1: Double = 3.20
    val trip2: Double = 4.10
    val trip3: Double = 1.72
    val totalTripLength: Double = trip1 + trip2 + trip3
    println("$totalTripLength miles left to destination")
}

```

Retourne 9.02

String

Concaténation avec '+'.

```

fun main() {
    val nextMeeting = "Next meeting:"
    val date = "January 1"
    val reminder = nextMeeting + date
    println(reminder)
}

```

Retourne Next meeting :January 1 »

Mettre des « » dans un string : \+ guillemets

Booléen :

```
fun main() {  
    val notificationsEnabled: Boolean = true  
    println(notificationsEnabled)  
}
```

Retourne true

```
fun main() {  
    val notificationsEnabled = 10.0  
    ! println(notificationsEnabled + "Are notifications enabled ?")  
}
```

! None of the following candidates is applicable: fun plus(other: Byte): Double
fun plus(other: Short): Double fun plus(other: Int): Double fun plus(other: Long): Double fun plus(other: Float): Double fun plus(other: Double): Double

Le « + » doit se faire entre deux nombres ou deux strings mais pas un nombre un string...

Conventions de codages :

space
↓
val discount: Double = .20
space
↓ ↓
var pet = "bird"
space
↓ ↓
val sum = 1 + 2

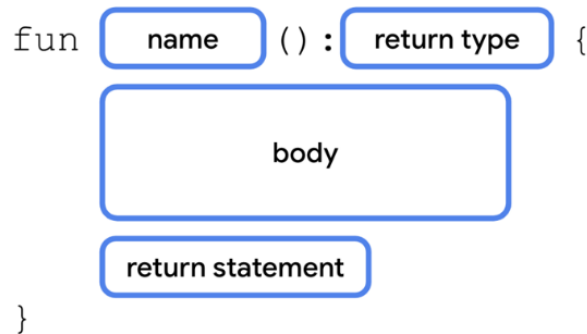
Il y'a une limite de 100 caractères pas lignes.

Summary

- A variable is a container for a single piece of data.
- You must declare a variable first before you use it.
- Use the `val` keyword to define a variable that is read-only where the value cannot change once it's been assigned.
- Use the `var` keyword to define a variable that is mutable or changeable.
- In Kotlin, it's preferred to use `val` over `var` when possible.
- To declare a variable, start with the `val` or `var` keyword. Then specify the variable name, data type, and initial value. For example: `val count: Int = 2`.
- With type inference, omit the data type in the variable declaration if an initial value is provided.
- Some common basic Kotlin data types include: `Int`, `String`, `Boolean`, `Float`, and `Double`.
- Use the assignment operator (`=`) to assign a value to a variable either during declaration of the variable or updating the variable.
- You can only update a variable that has been declared as a mutable variable (with `var`).
- Use the increment operator (`++`) or decrement operator (`--`) to increase or decrease the value of an integer variable by 1, respectively.
- Use the `+` symbol to concatenate strings together. You can also concatenate variables of other data types like `Int` and `Boolean` to `Strings`.

Créer et utiliser des fonctions en Kotlin

Définir le type de retour de notre fonction



Return type le permet

Unit est le type par défaut

```
fun main() {  
    birthdayGreeting()  
}  
  
fun birthdayGreeting(): Unit {  
    println("Happy Birthday, Rover!")  
    println("You are now 5 years old!")  
}
```

```
Happy Birthday, Rover!  
You are now 5 years old!
```

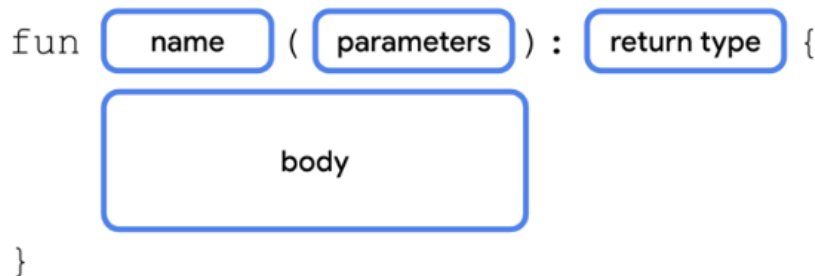
```
fun birthdayGreeting(): String {  
    ! val nameGreeting = "Happy Birthday, Rover!"  
    ! val ageGreeting = "You are now 5 years old!"  
    ! }  
}
```

Elle ne retourne rien 'Missing return statement »

Correction :

```
fun birthdayGreeting(): String {  
    val nameGreeting = "Happy Birthday, Rover!"  
    val ageGreeting = "You are now 5 years old!"  
    return "$nameGreeting\n$ageGreeting"  
}  
  
fun main() {  
    ! val greeting = birthdayGreeting() //N'affiche rien  
    println(birthdayGreeting()) //Affiche  
}
```

Ajouter des paramètres dans notre fonction



```
fun birthdayGreeting(name: String): String { //Paramètres name qui est un String
    val nameGreeting = "Happy Birthday, $name !"
    val ageGreeting = "You are now 5 years old!"
    return "$nameGreeting\n$ageGreeting"
}
fun main() {
    println(birthdayGreeting("Benoît"))
}
```

```
Happy Birthday, Benoît !
You are now 5 years old!
```

Signature d'une fonction

The diagram shows the function signature `fun birthdayGreeting(name: String, age: Int): String {` with labels and arrows: `name` points to `String`, `parameters` points to `name: String, age: Int`, and `return type` points to `String`. The body of the function is enclosed in curly braces, with a label `body` and an arrow pointing to the opening brace. A label `return statement` with an arrow points to the `return` statement inside the function body.

Arguments nommés :

Il est possible si jamais il y'a plus d'un paramètre dans notre fonction, de donner nos arguments des paramètres dans l'ordre voulu. Pas très important.

```
println(birthdayGreeting(name = "Rex", age = 2))
```

Valeurs par défaut :

```
fun birthdayGreeting(name: String = "Rover", age: Int): String {
    return "Happy Birthday, $name! You are now $age years old!"
}
fun main(){
    println(birthdayGreeting(age = 5))
    println(birthdayGreeting("Rex", 2))
}
```

```
Happy Birthday, Rover! You are now 5 years old!
Happy Birthday, Rex! You are now 2 years old!
```

Ici Rover est défini par défaut dans les paramètres

- Functions are defined with the `fun` keyword and contain reusable pieces of code.
- Functions help make larger programs easier to maintain and prevent the unnecessary repetition of code.
- Functions can return a value that you can store in a variable for later use.
- Functions can take parameters, which are variables available inside a function body.
- Arguments are the values that you pass in when you call a function.
- You can name arguments when you call a function. When you use named arguments, you can reorder the arguments without affecting the output.
- You can specify a default argument that lets you omit the argument when you call a function.

Mise en pratique : Bases de Kotlin

(A faire de notre côté)

Créer notre première application Android

```
class MainActivity : AppCompatActivity() { //ComponentActivity est un héritage
    override fun onCreate(savedInstanceState: Bundle?) {
        //onCreate qui est la fonction d'entrée (=main de l'application Android)
        super.onCreate(savedInstanceState)
        setContentView { //Définit le layout à travers les composants
            GreetingCardTheme {
                // A surface container using the 'background' color from the theme
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    Greeting("Android")
                }
            }
        }
    }
}
```

SetContent() s'occupe de l'interface graphique

Fonction greeting :

```
@Composable //Appel au composant "Composable"
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

Annotation @Preview : Donne un impact sur le preview uniquement, pas sur l'application finale

@Preview(showBackground= false) : Permet d'ajouter ou non un background à notre preview.

Annotation @Composable : Définit comme une fonction, permet d'indiquer que l'on va traiter l'interface graphique de notre application.

Changer la couleur de background

Il va falloir créer un composant qui va englober notre composant Text()

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hi, my name is $name!",
    )
}
```

Surround with Container
Surround with Row
Surround with Column

On va utiliser le composant « Surface() », c'est un **container** qui permet de stocker d'autres composants.

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Surface(color = Color.Cyan) { //Surface(color = Color.cyan (ou autre couleur))
        Text(
            text = "Hi, my name is $name!",
            modifier = modifier
        ) //Fermeture du composant Text
    } //Fermeture du container Surface
} //Fermeture de la fonction Greeting
```

Ajouter un padding :

Modifier : Utilisé pour décorer une composable, gère la mise en page

Padding : espace autour de notre élément, exemple de modifier

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Surface(color = Color.Cyan) {
        Text(
            text = "Hi, my name is $name!",
            modifier = modifier.padding(24.dp)
            //padding d'une taille de 24 pixels autour de notre élément
        )
    }
}
```

Dp = Density Pixel

Un composant text par défaut, fait juste ce qu'il y'a dans le bloc. Modifier va permettre d'appliquer des styles à cet objet graphique.

Il faut éviter de faire des modifications directement à l'intérieur du composable.

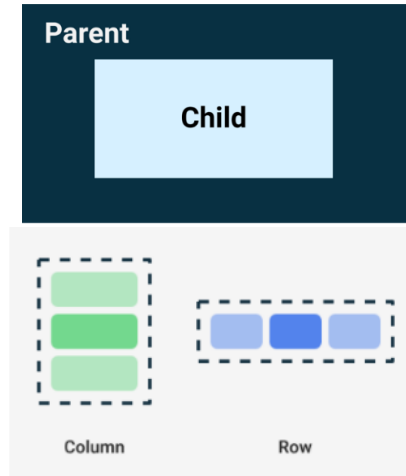
Summary

- To create a new project: open Android Studio, click **New Project** > **Empty Activity** > **Next**, enter a name for your project and then configure its settings.
- To see how your app looks, use the **Preview** pane.
- Composable functions are like regular functions with a few differences: functions names are capitalized, you add the `@Composable` annotation before the function, `@Composable` functions can't return anything.
- A `Modifier` is used to augment or decorate your composable.

Arranger nos textes par lignes ou colonnes

On va utiliser les composants 'row', 'columns' et 'box'

La hiérarchie de l'UI



Box va superposer nos éléments

- Jetpack Compose is a modern toolkit for building Android UI. Jetpack Compose simplifies and accelerates UI development on Android with less code, powerful tools, and intuitive Kotlin APIs.
- The user interface (UI) of an app is what you see on the screen: text, images, buttons, and many other types of elements.
- Composable functions are the basic building block of Compose. A composable function is a function that describes some part of your UI.
- The Composable function is annotated with the `@Composable` annotation; this annotation informs the Compose compiler that this function is intended to convert data into UI.
- The three basic standard layout elements in Compose are `Column`, `Row`, and `Box`. They are Composable functions that take Composable content, so you can place items inside. For example, each child within a `Row` will be placed horizontally next to each other.

Ajouter des images à notre application

Charger une image + compliquée qu'insérer du texte, car on a besoin de données externes.

View -> Tool Windows -> Resource Manager -> Drawable -> + -> Import Drawables

Density -> Non-Density

Insérer l'image dans notre application

On va utiliser la classe **R**. Il va contenir les IDs de toutes les ressources présentes dans le projet. L'ID de notre ressource est le même que le nom du fichier.

```
MyProject/  
  src/  
    MyActivity.kt  
  res/  
    drawable/  
      graphic.png  
    mipmap/  
      icon.png  
    values/  
      strings.xml  
  
R.drawable.graphic
```

On va utiliser la méthode **painterResource** qui va charger une image **drawable** et va prendre l'ID de l'image comme argument (donc le chemin pour accéder à l'image)

```
val image = painterResource(R.drawable.androidparty)
```

Après l'appel à **painterResource()** on rajoute un Composable **Image** et à l'intérieur on va y mettre un argument pour le **painter**.

```
Image(  
    painter = image  
)
```

ContentDescription va être mis en « »

Notre image et notre texte vont finir en superposé (l'image en dessous du texte) (car greetingimage était avant greetingtext)

Le Layout Box :

Ce layout va superposer des composants en fonction de ce que l'on souhaite.

Changer l'opacité et la scalabilité de notre image :

Utilisation du paramètre **ContentScale**.

```
Image(  
    painter = image,  
    contentDescription = null,  
    contentScale = ContentScale.Crop  
)
```

Crop va changer notre image afin de réguler sa taille en fonction de la dimension de l'écran.

```
Image(  
    painter = image,  
    contentDescription = null,  
    contentScale = ContentScale.Crop,  
    alpha = 0.5F  
)
```

Alpha change l'opacité

Modifier le layout de nos éléments

Utilisation de **Modifier**

```
// Example  
Text(  
    text = "Hello, World!",  
    // Solid element background color  
    modifier = Modifier.background(color = Color.Green)  
)
```

Le background ici va juste remplir en vert la zone de texte

Aligner et arranger le texte :

Utilisation de *Padding*. On avait utilisé jusqu'à présent un padding uniforme mais on peut faire en sorte de modifier le haut le bas...

```
// This is an example
Modifier.padding(
    start = 16.dp,
    top = 16.dp,
    end = 16.dp,
    bottom = 16.dp
)
```

Summary

- The **Resource Manager** tab in Android Studio helps you add and organize your images and other resources.
- An **Image** composable is a UI element that displays images in your app.
- An **Image** composable should have a content description to make your app more accessible.
- Text that's shown to the user, such as the birthday greeting, should be extracted into a string resource to make it easier to translate your app into other languages.